

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з дисципліни: «Програмування веб-застосунків»

Практична робота №8

Обробка подій та форми, клієнтська валідація

Виконав:

*Студент II курсу
групи ІІІ-56
Гончар І.В.*

Перевірив:

Ковтунець О.В.

1. Мета роботи:

Навчитися реагувати на дії користувача через `addEventListener` (`click`, `submit`, `input`, `change`), обробляти надсилання форми без перезавантаження сторінки (`event.preventDefault()`) і реалізувати клієнтську валідацію полів - вбудованими атрибутами HTML (`required`, `pattern`, `min/max`) і додатковою логікою JavaScript із власними повідомленнями про помилки. Форма додавання нового запису поповнюватиме масив даних і одразу перемальовуватиме список, розмічений через DOM у практикумі 7.

2. Хід виконання роботи

Крок 1–7. Додавання форми в HTML, зчитування та обробка сабміту:

У файл `index.html` додано секцію форми для створення нової гри (`#add-game-form`) із полями для назви, мін./макс. кількості гравців, а також елемент для виведення помилок валідації (`#form-error-msg`). У файлі `js/script.js` за допомогою `document.querySelector` обрано форму, додано обробник події `submit` із викликом `event.preventDefault()` для запобігання перезавантаженню сторінки, зчитано введені дані та реалізовано додавання нового об'єкта до масиву `games` через метод `.push()` із наступним викликом функції рендеру та очищення форми (`form.reset()`).

Крок 8. Додаткова клієнтська валідація (варіант №4):

Реалізовано перевірку перед додаванням елемента: якщо мінімальна кількість гравців перевищує максимальну (`minPlayers > maxPlayers`), виконання зупиняється, а в елемент `#form-error-msg` виводиться відповідне власне повідомлення про помилку.

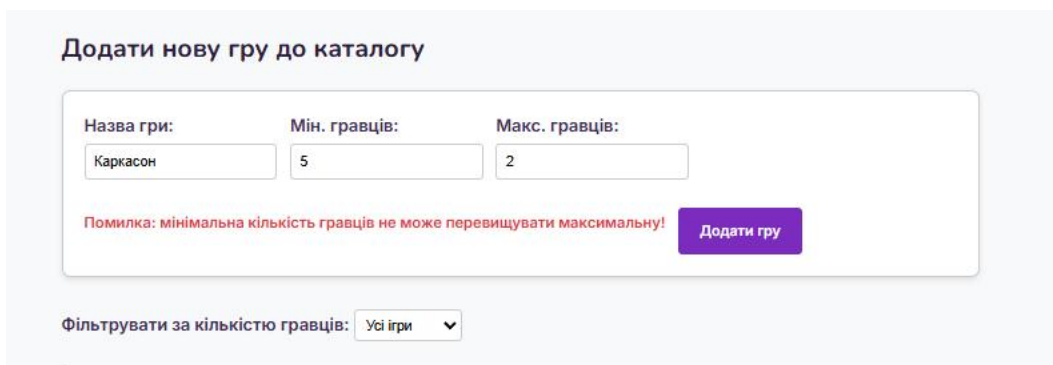


Рис. 1 — Перевірка клієнтської валідації форми (помилка при невірній кількості гравців)

Крок 9. Реалізація другої події та кросбраузерність:

Додано обробник події `change` на випадаючий список (`#player-filter`) для динамічної фільтрації списку ігор. Для перевірки кросбраузерності роботу форми та оновлення каталогу протестовано у браузерях Google Chrome та Opera GX. Результати відображення наведено нижче (рис. 2, 3, 4).

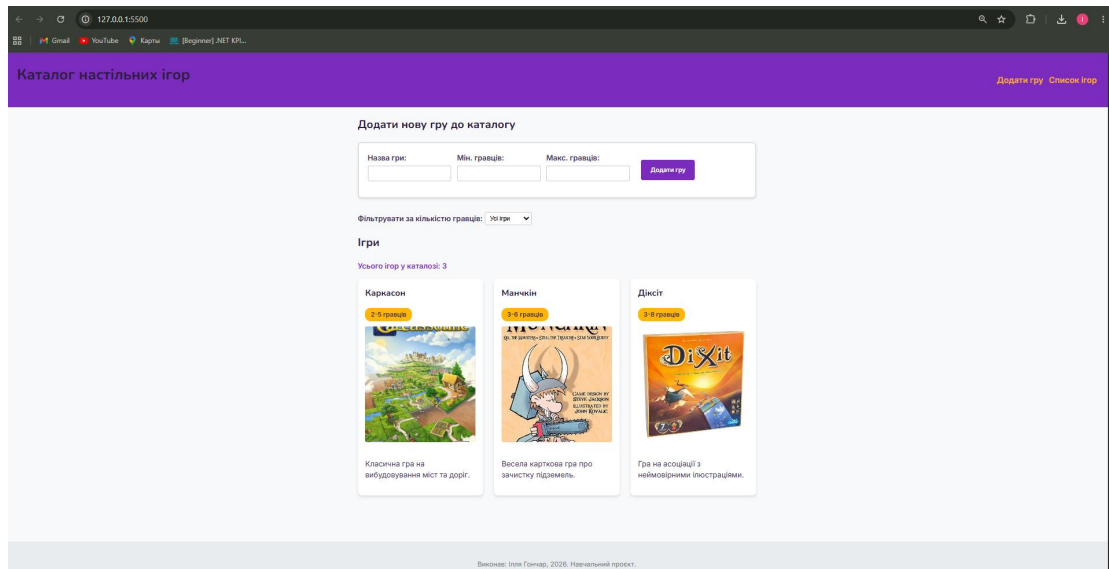


Рис. 2 — Перевірка роботи каталогу у браузері Google Chrome

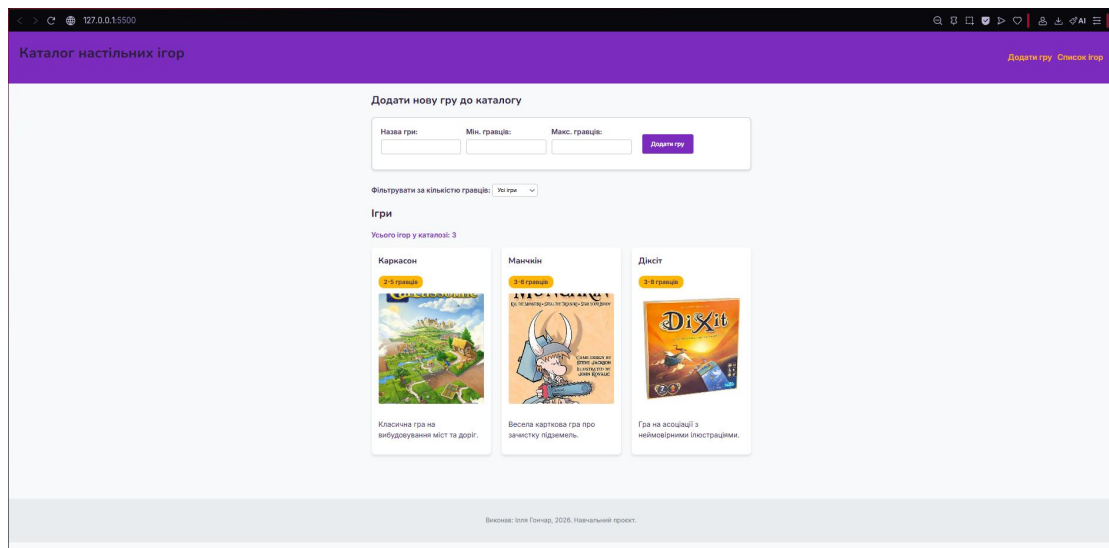


Рис. 3 — Перевірка роботи каталогу у браузері Opera GX

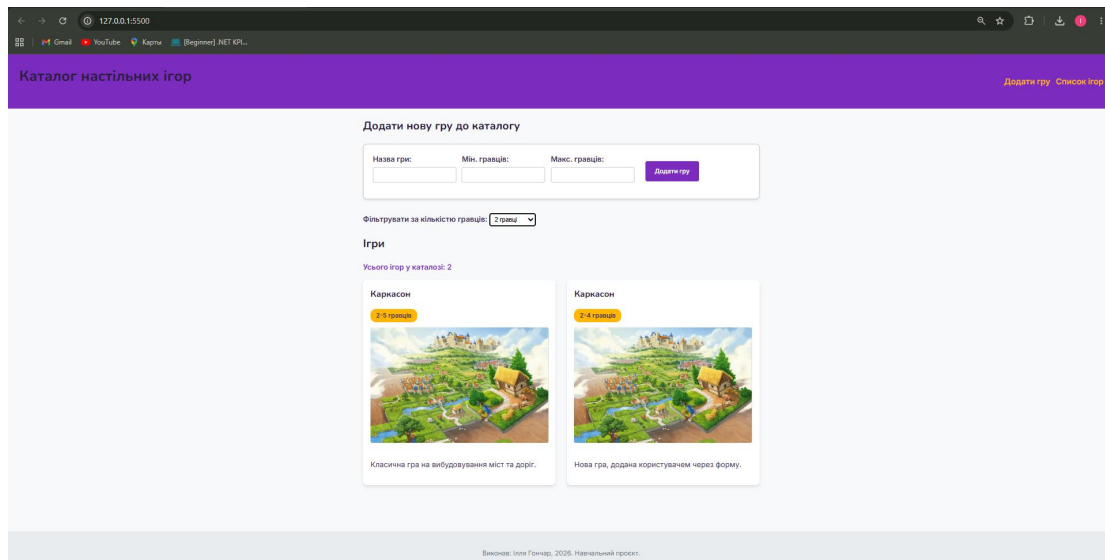


Рис. 4 — Загальний вигляд веб-застосунку після додавання елемента та оновлення лічильника

Крок 10–11. Перевірка відсутності перезавантаження та збереження на GitHub:

Перевірено відсутність перезавантаження сторінки під час сабміту та працездатність блокування некоректних даних. Усі зміни зафіксовано через систему контролю версій Git та опубліковано у віддаленому репозиторії.

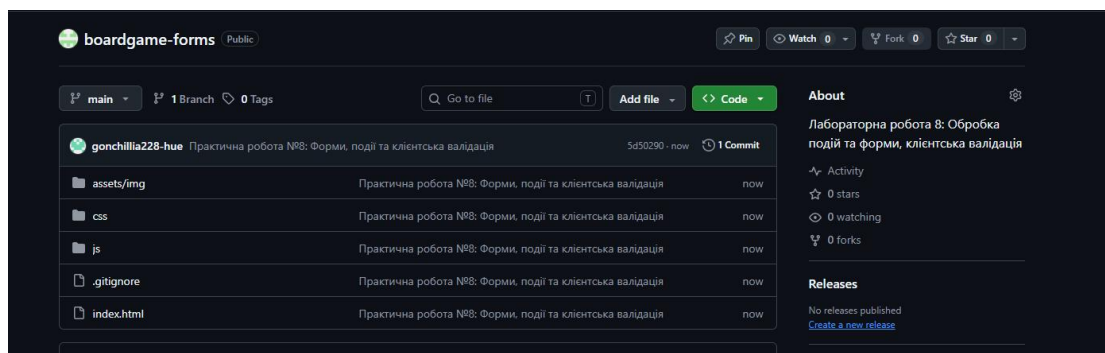


Рис. 5 — Фінальний коміт та проєкт у віддаленому репозиторії на GitHub

Посилання на репозиторій:

<https://github.com/Bitshkiper218/boardgame-forms>

Висновок:

Під час виконання практичної роботи було успішно засвоєно навички обробки подій за допомогою `addEventListener (submit, change)`, а також принципи запобігання стандартній поведінці браузера через `event.preventDefault()`. Реалізовано клієнтську валідацію полів форми з виведенням власних повідомлень про помилки у разі введення некоректних даних (зокрема перевірку співвідношення мінімальної та максимальної кількості гравців). Забезпечено повний інтерактивний цикл без перезавантаження сторінки: зчитування даних із форми, додавання їх до масиву об'єктів, миттєве оновлення дерева DOM за допомогою функції рендеру та очищення полів форми через `form.reset()`. Додатково реалізовано динамічну фільтрацію списку за допомогою другої події (`change`), а проєкт збережено в системі контролю версій Git та опубліковано на GitHub.